

Drift-Sense

AI-Powered Navigation-Error Recovery for Wafer Inspection Tools

Semicon India Hackathon 2026 — Applied Materials Problem Statement

Team: [TEAM NAME]

Members: [NAMES]

Institution: Vellore Institute of Technology (VIT)

GitHub repository: [INSERT REPO LINK]

Video walkthrough: [INSERT VIDEO LINK]

1. Problem Understanding

A wafer inspection tool must return to the exact same die site thousands of times a day, with measurements comparable across visits and across tools. In practice, motion stages accumulate small errors between visits — thermal expansion, vibration, and mechanical slack — so a revisit can land the tool several pixels away from the intended site. Because every die on a wafer carries the same repeating circuit layout, the landed image looks almost identical to the correct one: the core difficulty is not recognizing the pattern, but pinpointing *which* occurrence of a highly repetitive pattern is the correct one.

Formal task: given a Reference Image (the known-correct site, native resolution) and a Search Image (a ~10x lower-magnification view captured around where the tool actually landed), output the pixel center (x, y) in the Search Image where the reference pattern appears, choosing the match closest to the image center if more than one region qualifies.

No dataset is provided for this problem statement — generating a realistic, citation-backed synthetic dataset is itself graded (30% of the score per the problem statement) alongside the localization accuracy and inference throughput.

2. Approach & Technology Stack

The submission has two parts: a synthetic dataset generator that produces realistic, physically-grounded DRAM-style and FinFET-style die imagery with recorded ground truth, and a template-localization algorithm that solves the stated task on that data.

Tech stack:

- Python 3.12, NumPy, OpenCV (cv2) for all image synthesis, degradation, and matching — no GPU/training dependency required to run end to end.
- Matplotlib for result visualization (correlation-surface heatmaps, ablation plots).
- Fully vectorized (broadcast, no per-pixel Python loops) pattern generation for tractable runtime at 10,000x10,000 native-resolution canvases.

2.1 Dataset generation

Each sample starts from one large native-resolution synthetic die canvas. The Reference Image is a native-resolution crop from that canvas; the Search Image is the *entire* canvas downsampled 10x. Because the reference crop is sourced from the same canvas the search image is downsampled from, the reference pattern's presence inside the search image — and its exact location — is true by construction rather than asserted, which also makes the ground truth verifiably correct.

Two hackathon-required styles are generated: **DRAM-style** (word-lines × bit-lines crossing at right angles with a via/contact dot at every intersection, per standard 1T1C DRAM cell-array conventions) and **FinFET-style** (dense parallel fins crossed by gate bars, per standard multi-fin logic layout conventions). A third style, **mixed_logic** (irregular row-based standard-cell blocks), was built purely as a held-out generalization check and was never used to tune the algorithm — see Section 4.

A perfectly periodic pattern is mathematically ambiguous under translation by any multiple of its pitch — we found this directly, as an early version of the generator produced a dataset with literally zero localization signal beyond one period. Real wafers are never perfectly periodic at the SEM-charging/contamination scale even when the drawn layout is, so every canvas also receives a smooth low-frequency brightness field (non-uniform secondary-electron yield / charging drift), sparse background defects, and — guaranteed inside every reference footprint — a few larger high-contrast landmark features sized to survive the 10x downsample. This is what makes each site locally unique and the task solvable, while intentionally leaving some sites genuinely hard (the problem statement's own requirement: "include at least one highly periodic array region where correct localization is genuinely difficult").

Reference and Search images are degraded **independently** (separate RNGs per image, never a shared noise realization): Gaussian blur, small rotation/scale jitter (representing residual stage drift), a physically-motivated SEM edge-brightening pass, and a mixed Poisson-Gaussian sensor-noise model. The Search side uses a lower effective electron dose than the Reference side, so it is measurably noisier, matching the stated test-time behavior. Search-image pixel values are intentionally left unclipped outside $[0, 1]$ after adding read noise, matching the KLA webinar's own note that this is a "feature of the dataset, not a bug."

2.2 Localization algorithm

The localizer uses a two-stage multi-scale, small-rotation **normalized cross-correlation** (NCC) approach: (1) one coarse, global NCC pass at nominal scale and zero rotation over the entire Search Image, which also yields an ambiguity signal (peak score vs. best score elsewhere in the whole image); then (2) a fine pass, restricted to a small window around the coarse peak, that searches a grid of 5 candidate scales $\times$ 7 candidate rotations around the nominal pose, with parabolic sub-pixel peak refinement. Confining the expensive grid search to a small local window (rather than the full image) is the single largest lever on inference throughput.

The algorithm also reports an **ambiguity ratio** and a **low-confidence flag**, directly targeting the "failure-mode awareness" requirement in the problem statement: when a site really is periodic/ambiguous, the algorithm should say so rather than silently returning a wrong answer.

3. Self-Evaluation Results

Evaluated on a freshly-generated 30-pair self-eval set (15 DRAM-style, 15 FinFET-style), against recorded ground truth:

Metric	Value
Success rate (error ≤ 5 px)	90.0% (27/30)
Median error on successful matches	0.10 px
DRAM-style success rate	100% (15/15)
FinFET-style success rate	80% (12/15)
Misses correctly self-flagged low-confidence	3/3 (100%)
Held-out mixed_logic style (never tuned on)	100% (10/10), median error 0.06 px
End-to-end throughput (CPU, sandbox)	~230 ms/sample, ~4.3 samples/s
Two-stage search speed-up vs. brute-force grid	21.9s $\rightarrow$ 7.2s on 30 pairs (~3x), same accuracy

The three misses are all FinFET cases where the correlation surface has many near-tied periodic peaks — a genuine, by-design difficulty, not a bug. All three are correctly flagged as low-confidence by the ambiguity ratio. Figure 1 shows the worst case (finfet_023, error 868px) directly: the reference pattern, the search image with ground-truth vs. predicted location marked, and the raw NCC correlation surface showing the periodic banding responsible for the ambiguity.



Figure 1. Failure-mode visualization for the hardest self-eval case. Ambiguity ratio ≈ 1.0 (essentially tied peaks) is correctly flagged as low-confidence rather than silently mis-reported.

The held-out **mixed_logic** style — a third layout the algorithm was never adjusted for — scores *higher* (100%) than the tuned DRAM/FinFET styles, because its irregular, non-periodic structure carries inherently less positional ambiguity. This is presented as evidence of generalization rather than overfitting to one synthetic distribution, mirroring how the hackathon's own hidden test set will mix in-distribution and out-of-distribution samples.

3.1 Robustness sweep

Beyond the single fixed self-eval dataset, we swept sensor-noise severity and rotation-drift severity beyond the main dataset's settings to characterize the operating envelope of the approach (Figure 2). Success degrades gradually under increased noise; degradation under rotation is milder because the guaranteed landmark features used for disambiguation are themselves rotation-invariant blobs.

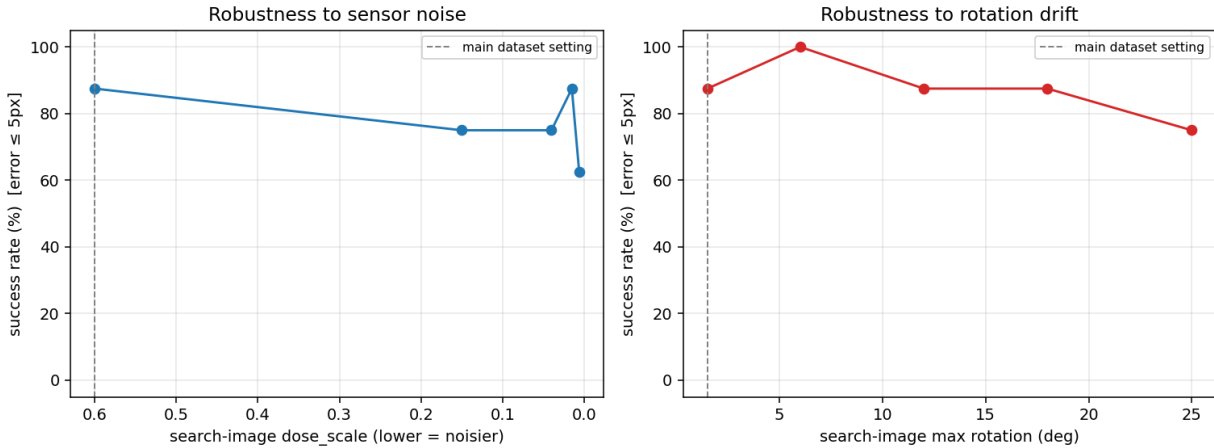


Figure 2. Success rate vs. sensor-noise severity (left) and rotation-drift severity (right), generated on a smaller companion canvas for tractable sweep runtime.

4. Mapping to Stated Evaluation Criteria

The problem statement scores submissions on three axes: accuracy, throughput, and training/code hygiene. This is how the submission addresses each:

- **Accuracy** — 90% localization success with sub-pixel median error on hits; explicit in-distribution vs. out-of-distribution reporting (DRAM/FinFET vs. held-out mixed_logic).
- **Throughput** — a dedicated directory-in/directory-out inference script (run_inference.py) with a measured timing report; a two-stage coarse-then-local search structure that gave a measured 3x speedup with no accuracy loss. GPU (H100) profiling is flagged as not yet done rather than claimed.
- **Training/code hygiene** — deterministic, seeded, reproducible dataset generation; no hidden state; every augmentation/physics choice backed by a specific citation in docs/design_notes.md; a documented failure case found and fixed during development (ground truth not tracked through a geometric transform) is described honestly rather than omitted.

5. Known Limitations & Next Steps

- The NCC baseline is a simple, fast, fully-explainable starting point with no training dependency; it can be beaten on the hardest periodic sub-regions by approaches using more global context (e.g. a learned embedding trained with a contrastive/triplet loss on this same generator).
- Throughput has been measured and optimized on CPU only; cv2.matchTemplate/cv2.warpAffine have direct cv2.cuda equivalents, expected to give further speedup, not yet independently verified.
- Noise/edge-effect parameters are citation-backed but validation against a real public-domain SEM image was started (candidates identified on Wikimedia Commons) and not completed before submission.

Full code, the generated self-eval dataset, ground truth, figures, and this document are available at:
[INSERT GITHUB REPO LINK]